

Lab #3

Group number : 5

Group members : Aditya Ray - 1233750715

Krutyanjay Shinde - 1234108189

PRE-LAB

1. The I²C bus on the RP2040 is accessed by reading and writing memory addresses beginning at 0x40044000. Does this use AHB, APB, or IOPORT?
 - We are using the I2C0 which is part of **APB peripherals**. Which start at base address of 0x40000000

Table 15. Address Map Summary

ROM	0x00000000
XIP	0x10000000
SRAM	0x20000000
APB Peripherals	0x40000000
AHB-Lite Peripherals	0x50000000
IOPORT Registers	0xd0000000
Cortex-M0+ internal registers	0xe0000000

APB Peripherals:

SYSINFO_BASE	0x40000000
SYSCFG_BASE	0x40004000
CLOCKS_BASE	0x40008000
RESETS_BASE	0x4000c000
PSM_BASE	0x40010000
IO_BANK0_BASE	0x40014000
IO_QSPI_BASE	0x40018000
PADS_BANK0_BASE	0x4001c000
PADS_QSPI_BASE	0x40020000
XOSC_BASE	0x40024000
PLL_SYS_BASE	0x40028000
PLL_USB_BASE	0x4002c000
BUSCTRL_BASE	0x40030000
UART0_BASE	0x40034000
UART1_BASE	0x40038000
SPI0_BASE	0x4003c000
SPI1_BASE	0x40040000
I2C0_BASE	0x40044000
I2C1_BASE	0x40048000
ADC_BASE	0x4004c000
PWM_BASE	0x40050000
TIMER_BASE	0x40054000
WATCHDOG_BASE	0x40058000
RTC_BASE	0x4005c000

2. To understand how a program interacts with hardware, it is important to understand exactly how the C language uses pointers.

- a. Which of the following code snippets reads the value of a 32-bit register at address 0x40001000 into a variable named foo?

Answer :-

```
// (2)
uint32_t *load = (uint32_t *) 0x40001000;
uint32_t foo = *load;
```

```
// (2)
uint32_t *load = (uint32_t *) 0x40001000;
uint32_t foo = *load;
```

- b. Which of the following code snippets writes the value 5 to the address 0x4000600F?

Answer :-

```
// (3)
uint32_t *store = (uint32_t *) 0x4000600F;
*store = 5;
```

```
// (3)
uint32_t *store = (uint32_t *) 0x4000600F;
*store = 5;
```

3.

- a. Which bus, AHB, APB, or IOPORT, is used to control the GPIO pin that turns on an off the LED?
- The specific bus used to communicate with a hardware peripheral is determined by the peripheral's memory address.
 - The provided code and information show that turning the LED on or off involves writing to the `sio_hw->gpio_set` or `sio_hw->gpio_clr`

registers. The base address for this hardware block (`sio_hw`) is **0xd0000000**.

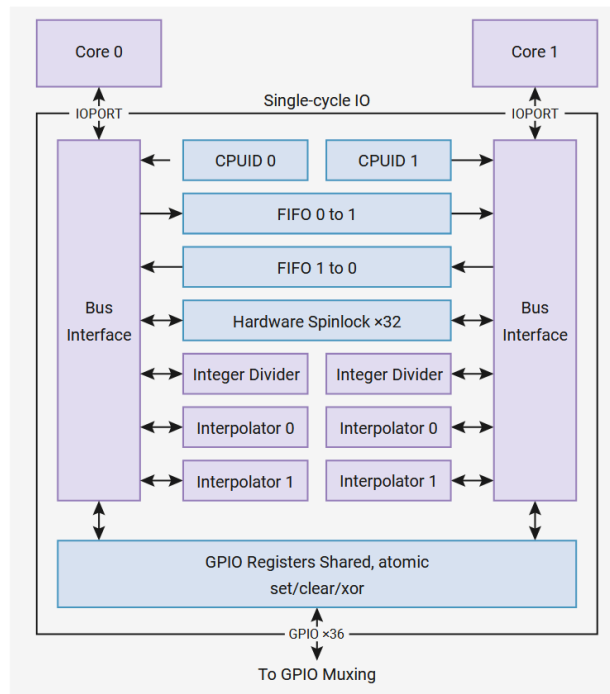
Table 15. Address Map Summary

ROM	0x00000000
XIP	0x10000000
SRAM	0x20000000
APB Peripherals	0x40000000
AHB-Lite Peripherals	0x50000000
IOPORT Registers	0xd0000000
Cortex-M0+ internal registers	0xe0000000

According to the RP2040 datasheet's memory map, the address range starting at **0xd0000000** is reserved for the **SIO (Single-cycle I/O)** block.

- The SIO block is directly connected to each processor core through the **IOPORT bus**, bypassing the slower, more general-purpose AHB and APB buses. This dedicated connection allows for extremely fast, single-cycle access to the GPIOs.

Figure 7. The single-cycle IO block contains memory-mapped hardware which the processors must be able to access quickly. The FIFOs and spinlocks support message passing and synchronisation between the two cores. The shared GPIO registers provide fast and concurrency-safe direct access to GPIO-capable pins. Some core-local arithmetic hardware can be used to accelerate common tasks on the processors.



- So the answer is IOPORT bus.

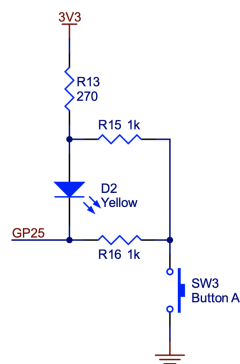
- b. Explain the role of the mask variable in `gpio_put`. Why is its value the same whether you are turning on or off the LED? How does the hardware recognize whether we intend to turn on or off the LED?

- The **mask** variable is calculated using the expression `1UL << gpio` inside the `gpio_put` function, where `gpio` is the number of the specific pin we want to control. This creates a value where only the bit corresponding to that pin's position is a '1', and the rest of the bits are zeros.

```
static inline void gpio_put(uint gpio, bool value) {
    #if PICO_USE_GPIO_COPROCESSOR
        gpio_bit_out_put(gpio, value);
    #elif NUM_BANK0_GPIOS <= 32
        uint32_t mask = 1ul << gpio;
        if (value)
            gpio_set_mask(mask);
        else
            gpio_clr_mask(mask);
    }
```

- So essentially we are telling the microcontroller not to change the value of any other bit in the register other than the one we have indicated using a mask.
- This mask value is the same whether we turn the LED **on or off**. That's because the mask doesn't actually set the pin's state; it just tells the microcontroller **which pin** we want to use.

- c. The second argument to `gpio_put` specifies whether to "set" or "clear" the GPIO pin. Setting the pin means bringing its voltage high, whereas clearing it means bringing the voltage low (to ground). The 3pi 2040 Control Board Schematic reveals that GPIO pin 25 is connected on the robot as follows:



Should you set or clear the GPIO pin to turn on the LED? What do you expect

will happen when you push button A?

- We need to **clear** the GPIO pin to turn ON the LED.
- Considering the GP25 to be High z, if we press SW3 the LED will turn on but dimmer, as compared to the condition in which GPIO is set to clear.

Checkoff

1. Controlling GPIO using memory mapped IO

```
target C {
    platform: {
        name: "rp2040",
        board: "pololu_3pi_2040_robot"
    },
    single-threaded: true
}

preamble {=
    #include <stdio.h>
    #include <pico/stdlib.h>
    #include <hardware/gpio.h>
=}

main reactor {
    timer t(0, 250 ms)
    state led_on: bool = false

    reaction(startup) {=
        gpio_init(PICO_DEFAULT_LED_PIN);
        gpio_set_dir(PICO_DEFAULT_LED_PIN, GPIO_OUT);
    =}

    reaction(t) {=
        volatile uint32_t *led_sio_set = 0xd0000000 + 0x00000014;
        volatile uint32_t *led_sio_clr = 0xd0000000 + 0x00000018;
        self->led_on = !self->led_on;
        printf("LED State: %b\n", self->led_on);
        if (!self->led_on)
            *led_sio_set = (1ul << PICO_DEFAULT_LED_PIN);
        else
            *led_sio_clr = (1ul << PICO_DEFAULT_LED_PIN);
    =}
```

```

    //gpio_put(PICO_DEFAULT_LED_PIN, !self->led_on);
    =}
}

```

We first set the pin direction to be output in the startup reaction. In the reaction I have stored the address in `led_sio_set` for the `GPIO_SET` register which is at `0xd0000014` and `led_sio_clr` for `GPIO_CLR` register which is at `0xd0000018` as volatile `uint32_t` pointers. We then flip the state and create a mask of 1 at `PICO_DEFAULT_PIN` index and assign it as a value in the set register to set it and clear register to clear it. This reaction is triggered every 250ms.

2. Pooling input

```

target C {
    platform: {
        name: "rp2040",
        board: "pololu_3pi_2040_robot"
    },
    single-threaded: true
}

import Display from "lib/Display.lf"

preamble {=
    #include <stdio.h>
    #include <pico/stdlib.h>
    #include <hardware/gpio.h>
=}

main reactor {
    timer t(0, 250 ms)
    d = new Display()

    reaction(startup) {=
        gpio_init(PICO_DEFAULT_LED_PIN);
        gpio_set_dir(PICO_DEFAULT_LED_PIN, GPIO_IN);
    =}

    reaction(t) -> d.line0 {=
        bool state = false;
        static char buff[17];
        volatile uint32_t *led_sio_in = 0xd0000000 + 0x00000004;
    =}
}

```

```

state = *led_sio_in & (1ul << PICO_DEFAULT_LED_PIN);
snprintf(buff, 17, "LED State: %b", !state);

lf_set(d.line0, buff);
//gpio_put(PICO_DEFAULT_LED_PIN, !self->led_on);
= }
}

```

We first set the GPIO pin to be input. Then create a volatile uint32_t pointer to store the address for GPIO_IN which is 0xd0000004 and AND it with the desired value to check if we get a HIGH or LOW. We also have a 17 byte buffer to store the string to be printed and we use the [Display.lf](#) to display it on line0 of the display.

Postlab

- Address 0xd000001c is defined in Section 2.3.1.7, List of Registers, of the RP2040 Datasheet as GPIO_OUT_XOR or "GPIO output value XOR." Writing a mask to this address will, in one cycle, reverse the polarity of the GPIO pin; if it is set, it will be cleared, and if it is cleared, it will be set. Why do you think the hardware designers chose to provide this functionality? If this functionality were not available, how would you reverse the polarity of a pin? Is there a risk that your alternative solution would not cause anything to change on the pin?
 - In embedded systems, we often need to toggle GPIO pins—one of the most common use cases being generating clock signals.
 - Usually, we'd do this in software. But a true toggle isn't as simple as just setting a pin high then low; it's a three-step "read-modify-write" process:
 - Read the current state of the pin (is it on or off?).
 - Modify the state in the CPU (use a NOT operator to invert).
 - Write that new, opposite state back to the pin.
 - We expect the LED to toggle with perfect timing, but that's not what happens, each of those steps takes up a few clock cycles, and that delay gets added to our calculation. The entire three-step process is not atomic—meaning it can be interrupted.
 - The software can be non deterministic if there are concurrent processes running or two processors running the code in parallel, so the exact time at which the toggle happens is unpredictable.
 - So, that's why the hardware designers gave us the GPIO_OUT_XOR feature. It does that entire read-modify-write sequence in a single, uninterruptible hardware operation. It's atomic and guaranteed to finish in one clock cycle, so no interrupt

can mess it up. For situations where we need to guarantee the time for a toggle and ensure it's safe.

2. You need to change the state of two GPIO pins, and you want them to change at the same time - closer together in time than the clock period of the processor. To accomplish this, you try to write both of their corresponding bits in the GPIO OUT register in the same write function. Do you think this will cause both pins to switch at the same time? Why or why not?
 - YES. Setting a single 32-bit value to the GPIO_OUT register will cause multiple pins to switch their states simultaneously for all practical purposes.
 - BUT. While the update is logically parallel, it is not perfectly simultaneous at a physical time due to minor, unavoidable hardware delays.
3. The mask functionality explored in this lab may seem a bit unusual. Suppose that instead of providing a mask, when you want to change the state of a GPIO pin you had to write a single 32 bit number to a memory-mapped register, where the value you write specifies the state of all the GPIO pins. What would be the potential pitfalls of such a mechanism?
 - The primary purpose of a mask is to modify a specific pin without affecting any other pins that we don't intend to change. If this mask-based system were not available and we had to write the full 32-bit value to the register for every change, there would be significant pitfalls.
 - We would either have to manually calculate the correct 32-bit value for every single change, which is prone to human error, or we would have to write code to perform the calculation. While setting a given pin one might accidentally set or clear some other pin which should not be touched by that code. It is faster in hardware than performing such operation in software where we use some cycles to create the mask before applying it.
4. What were your takeaways from the lab? What did you learn during the lab? Did any results in the lab surprise you?
 - So, the most important thing I figured out in this lab is that the functions I used in libraries like Arduino weren't giving me the exact delay they promised because of software overhead. It turns out the function call itself takes up clock cycles, so the real delay you get is the time you asked for plus a little extra from the code itself running, which can mess things up when timing is critical.
 - We learnt about how memory mapped IOs are used in embedded systems. The microcontroller also provides mask registers which is really convenient for setting, clearing and toggling the GPIO pins. The toggle functionality was new to us.
 - I also found out that's why there are different buses for different peripherals. It's not just that some are fast and some are slow; they are designed that way on

purpose. You get a fast bus for the important stuff that needs speed and a slower, simpler bus for basic peripherals that don't. It's all a trade-off to make the chip work efficiently.

References:

rp2040-datasheet.pdf